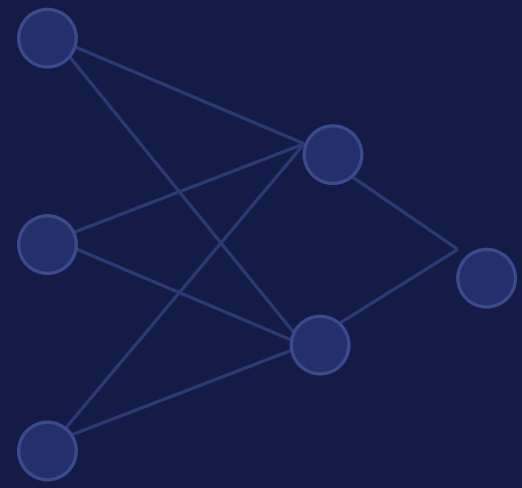
КУРС «НЕЙРОННЫЕ СЕТИ»

Тема 2. Оптимизаторы и



Лекция 3 · Модуль 1

ПЛАН ЛЕКЦИИ

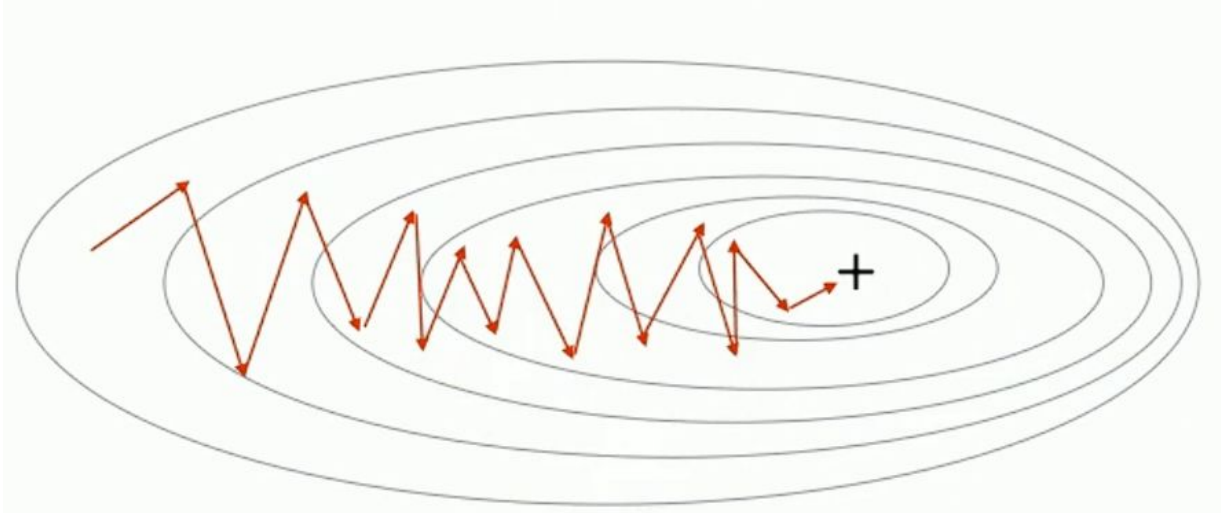
О чём сегодня говорим

1. **Правило цепочки и вычислительный граф**
   математическая база backprop
2. **Прямой и обратный проход**
   считаем вручную, сверяем с PyTorch
3. **От ручных циклов к nn.Module**
   Linear, Sequential, ReLU
4. **Многоклассовая классификация**
   softmax и cross-entropy
5. **torch.optim и мини-батчи**
   DataLoader, эпохи, батчи
6. **MNIST: собираем и обучаем сеть**
   полный цикл целиком

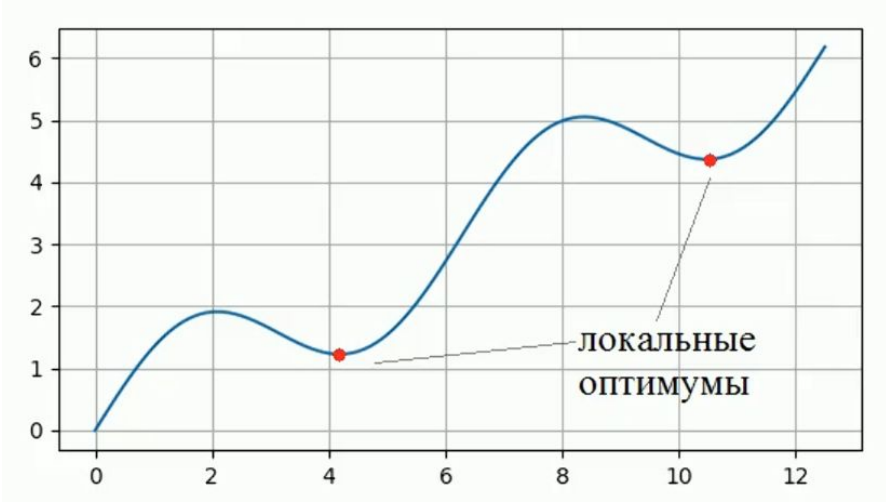
Минусы обычного SGD

Издержки псевдоградиента

Псевдаградиенту характерны “скачки” - численная нестабильность связанная с тем, что градиент высчитывается каждый раз заново



В обычном спуске могут случаться застревания в локальной точке минимума (оптимума)

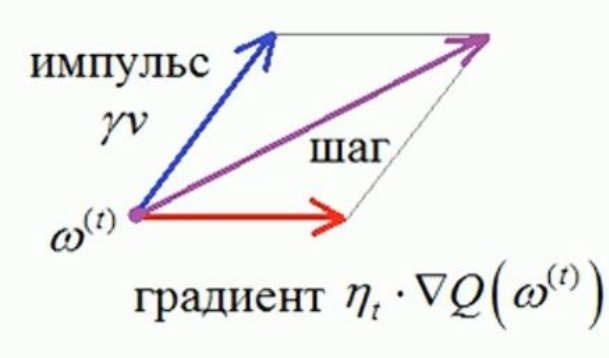


Метод импульсов (momentum) и импульс Нестерова (NAG - Nesterov's accelerated gradient)

Усредняем градиент по шагам, и добавляем постепенное затухание предыдущих значений:

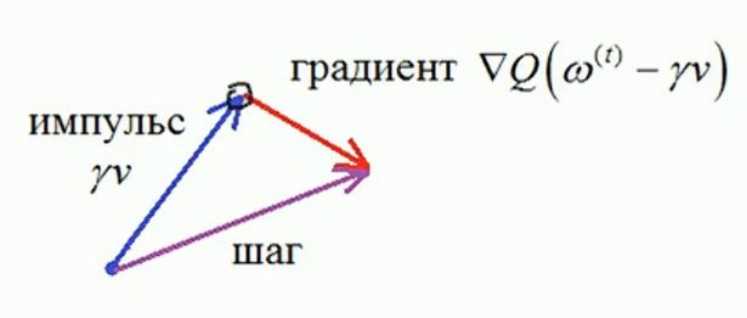
$$v = \gamma v + (1-\gamma)\eta_t \cdot \nabla Q\left(\omega^{(t)}\right)$$

$$\omega^{(t+1)} = \omega^{(t)} - v$$



В импульсе Нестерова сразу смещаемся:

$$v = \gamma v + \eta_t \cdot \nabla Q\left(\omega^{(t)} - \gamma v\right)$$



Как бороться

RMSProp и Adam

Усредняем градиент по шагам, и добавляем постепенное затухание предыдущих значений:

$$\frac{dL_i(\omega)}{d\omega} = \begin{bmatrix} \partial L_i(\omega_1)/\partial\omega_1 \\ \dots \\ \partial L_i(\omega_n)/\partial\omega_n \end{bmatrix}$$

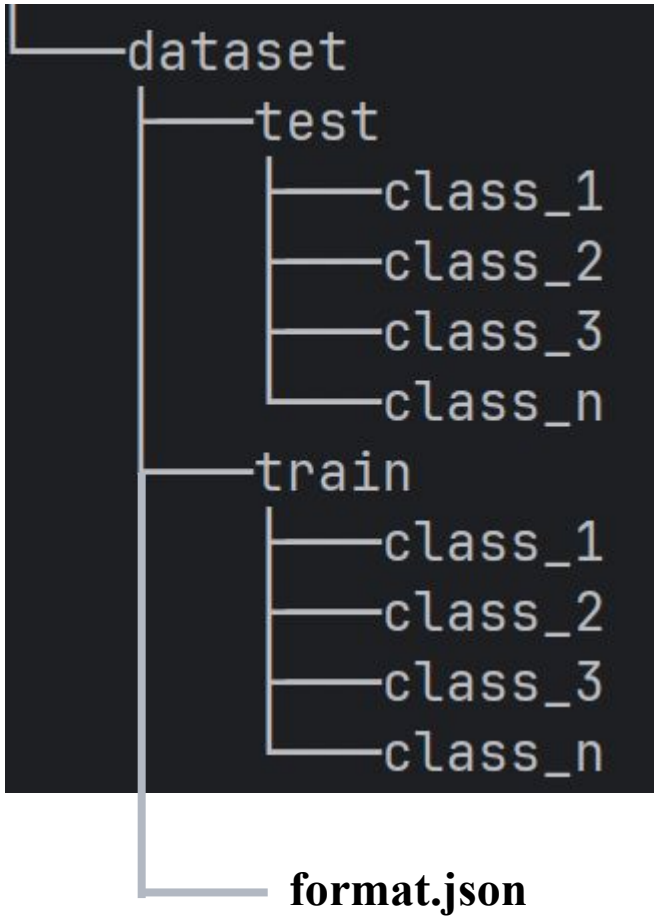
$$v = \gamma v + (1-\gamma)\nabla L_i\left(\omega^{(t)}\right)$$
$$G = \alpha G + (1-\alpha)\nabla L_i\left(\omega^{(t)}\right) \odot \nabla L_i\left(\omega^{(t)}\right)$$

$$G = \alpha G + (1-\alpha)\nabla L_i\left(\omega^{(t)}\right) \odot \nabla L_i\left(\omega^{(t)}\right)$$

$$\omega^{(t+1)} = \omega^{(t)} - \eta \cdot \frac{\nabla L_i\left(\omega^{(t)}\right)}{\sqrt{G} + \varepsilon}$$

$$\omega^{(t+1)} = \omega^{(t)} - \eta \frac{\hat{v}}{\sqrt{\hat{G}} + \varepsilon}$$

Разделение



format.json

```
{
    “class_1”: 1
    “class_2”: 2
    “class_3”: 3
    “class_4”: 4
    …
}
```

format.json

```
{
    “png_1”: [113, 131]
    “png_2”: [123, 186]
    …
}
```

format.json **соотносит** изображение с **меткой класса** (классификация), либо с **координатами местоположения** искомого объекта (регрессия)

Выборка

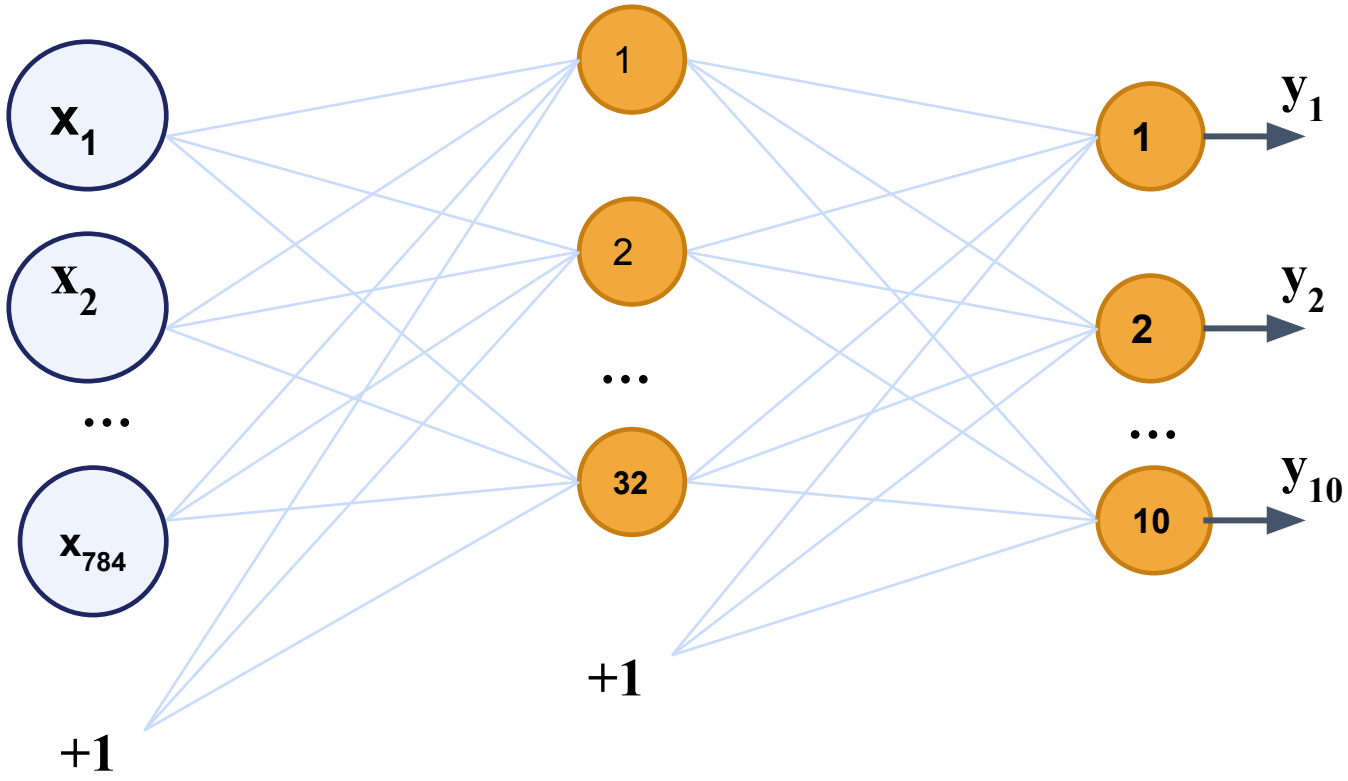
Баланс выборок и репрезентативность

1. Если у одного из классов значительно превалирует количество образов, то модель может начать всегда выдавать метку этого класса и в среднем мало ошибаться

2. Образы в обучающей выборке должны быть как можно более разнообразны, чтобы модель имела высокие обобщающие способности



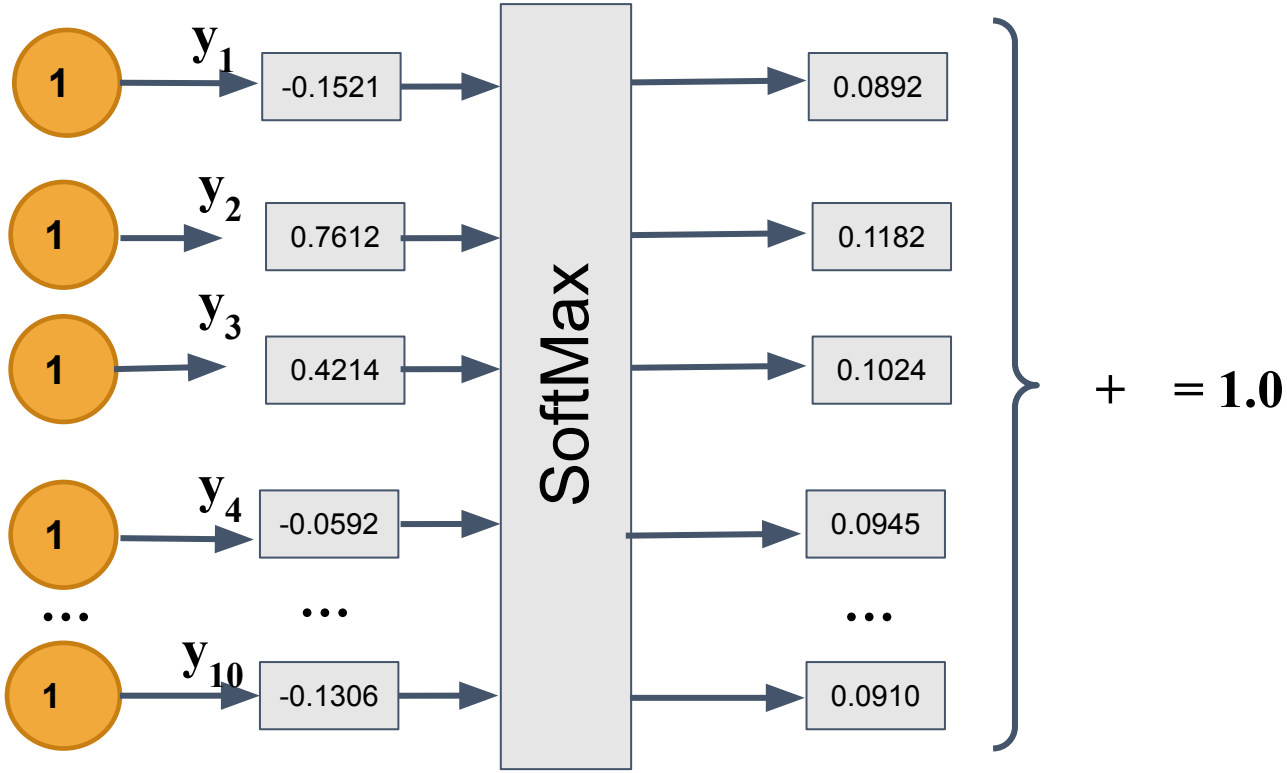
Структура нейронной сети



1	0	0	0
0	0	0	1
0	0	0	0
0	0	0	0
0	0	1	0
0	1	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

batch

Структура нейронной сети

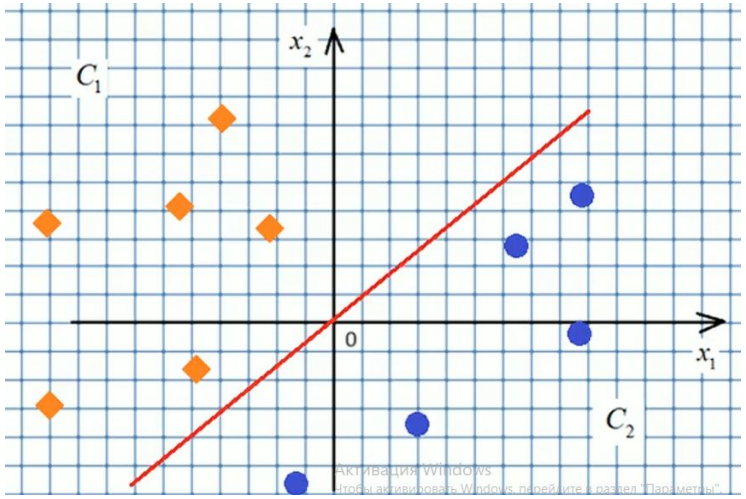



```
loss_function = nn.CrossEntropyLoss()

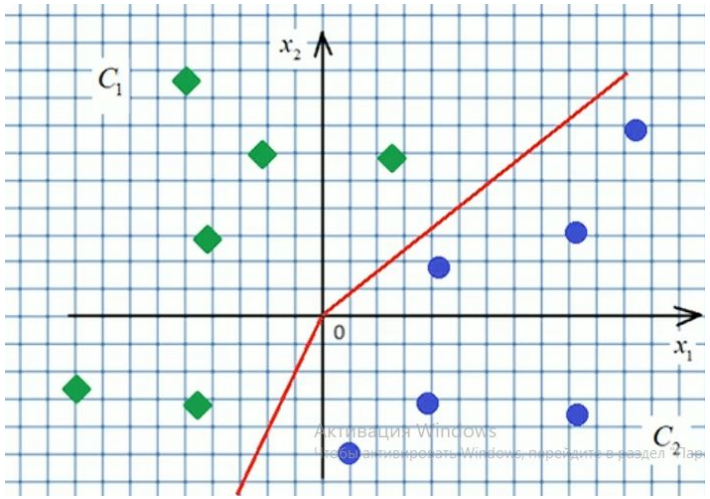
loss = lss_function(predict, y_train)
```

Слишком много нейронов

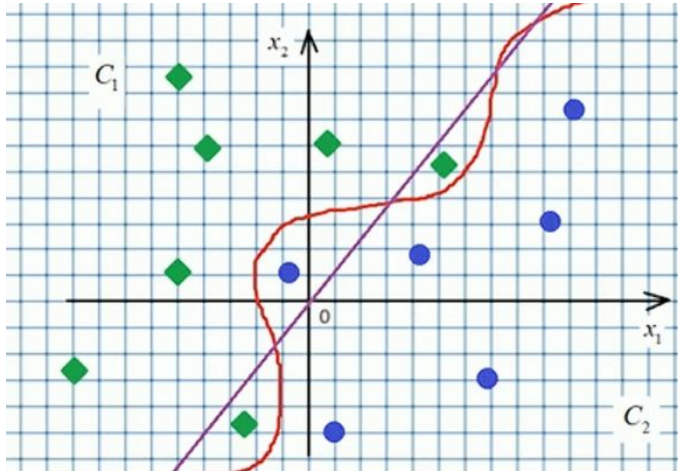
Один нейрон



Два нейрона



Много нейронов



Выборка валидации

Обучающая выборка

Выборка валидации

Тестовая выборка

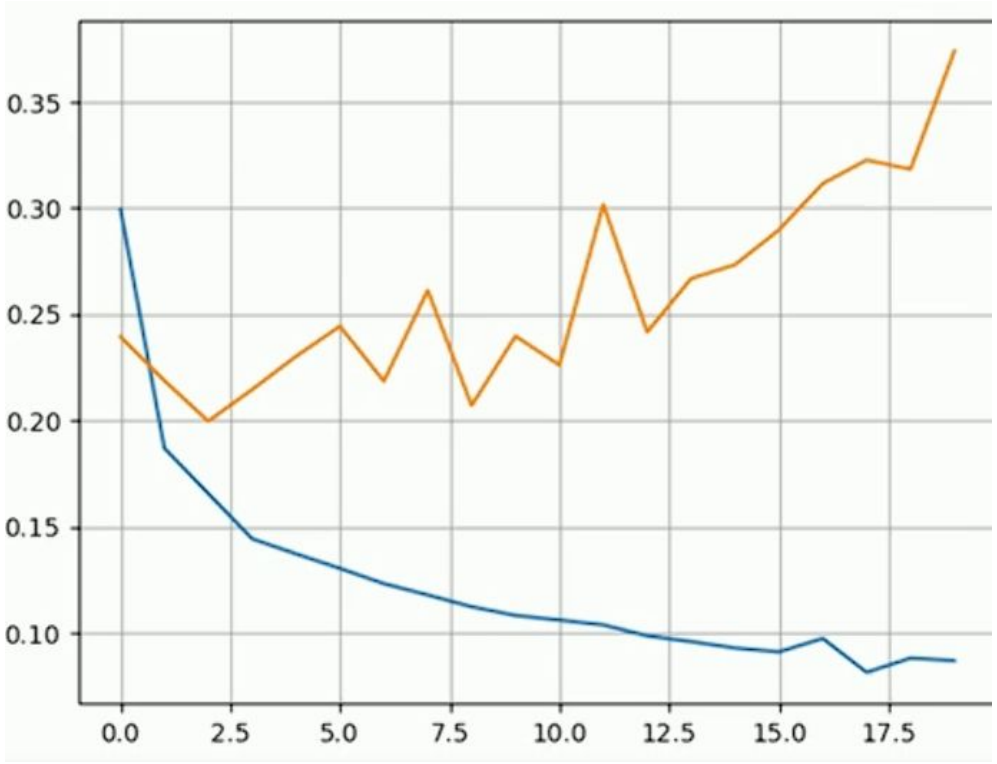
Критерии останова:

1. Расхождение показателя качества для обучающей выборки и выборки валидации
2. от итерации к итерации показатель качества Q практически не меняется
3. Малое изменение весовых коэффициентов
4. Достижение максимального количества итерации

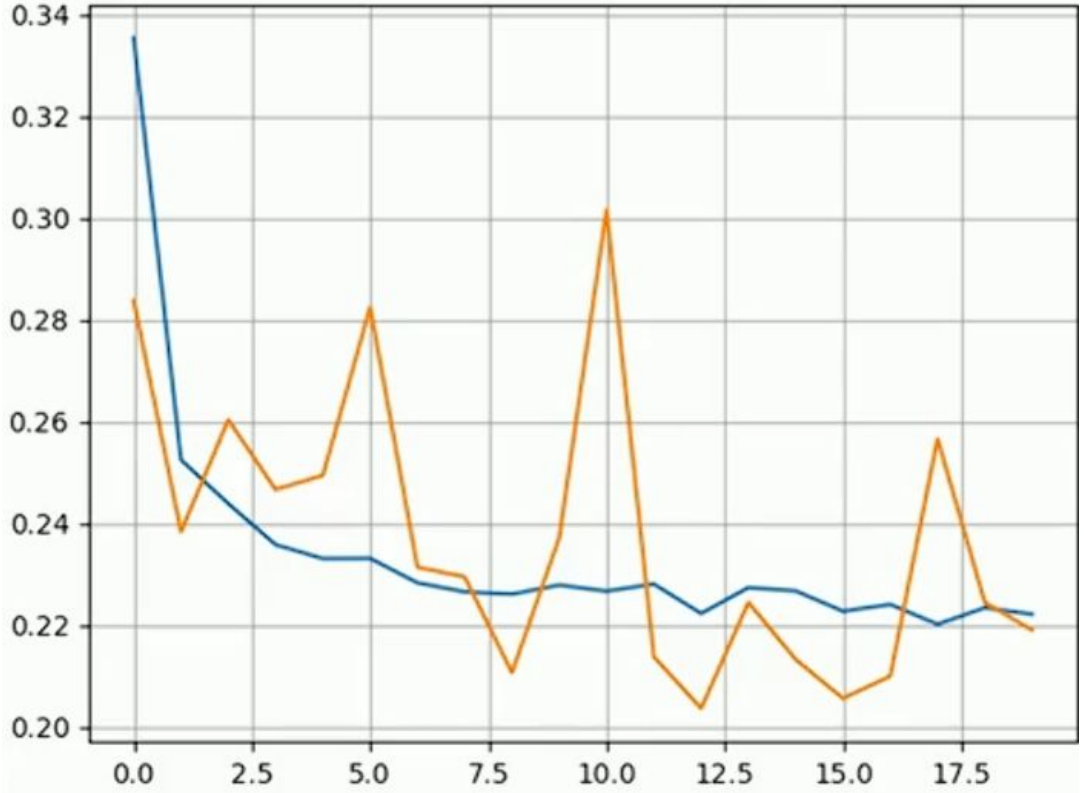


Регулизатор

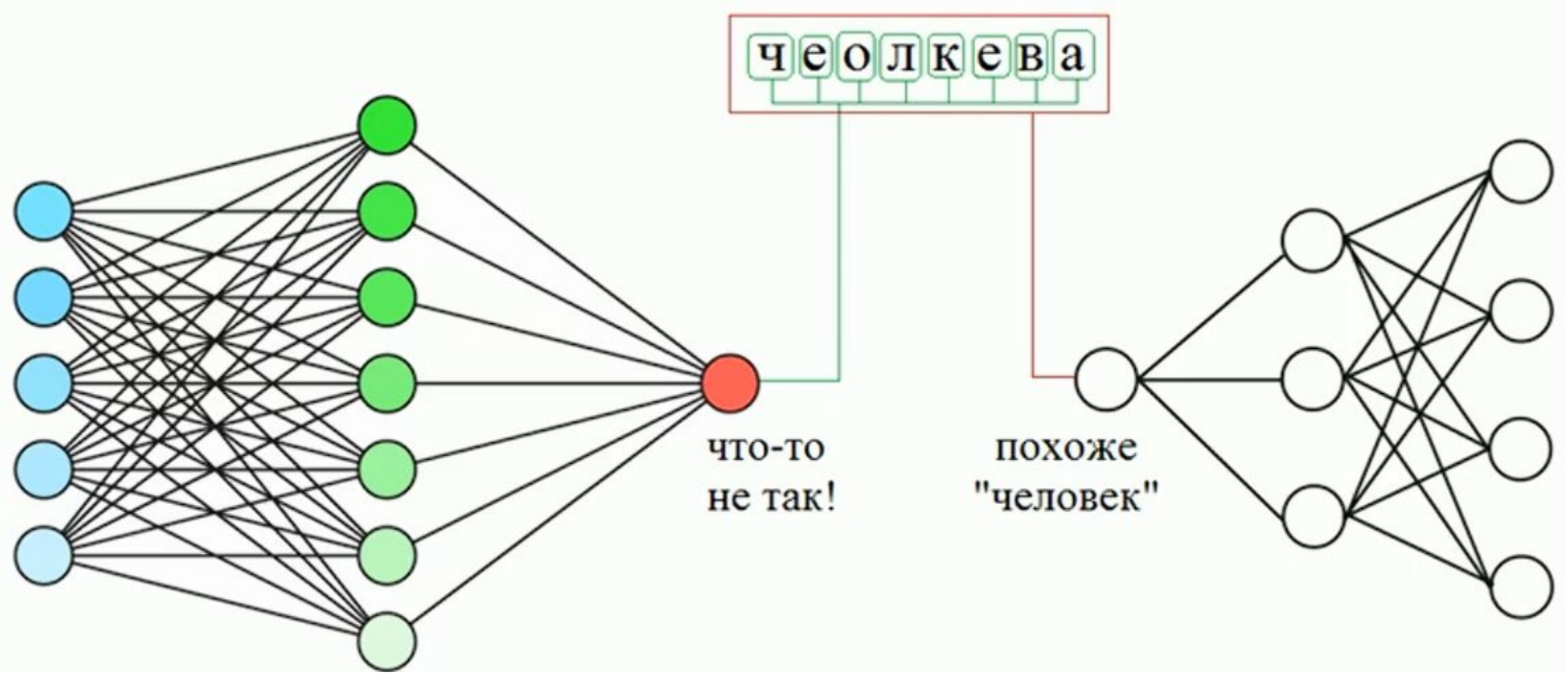
Без L2-регуляризации:
optim.Adam(params=model.parameters(), lr=0.01)



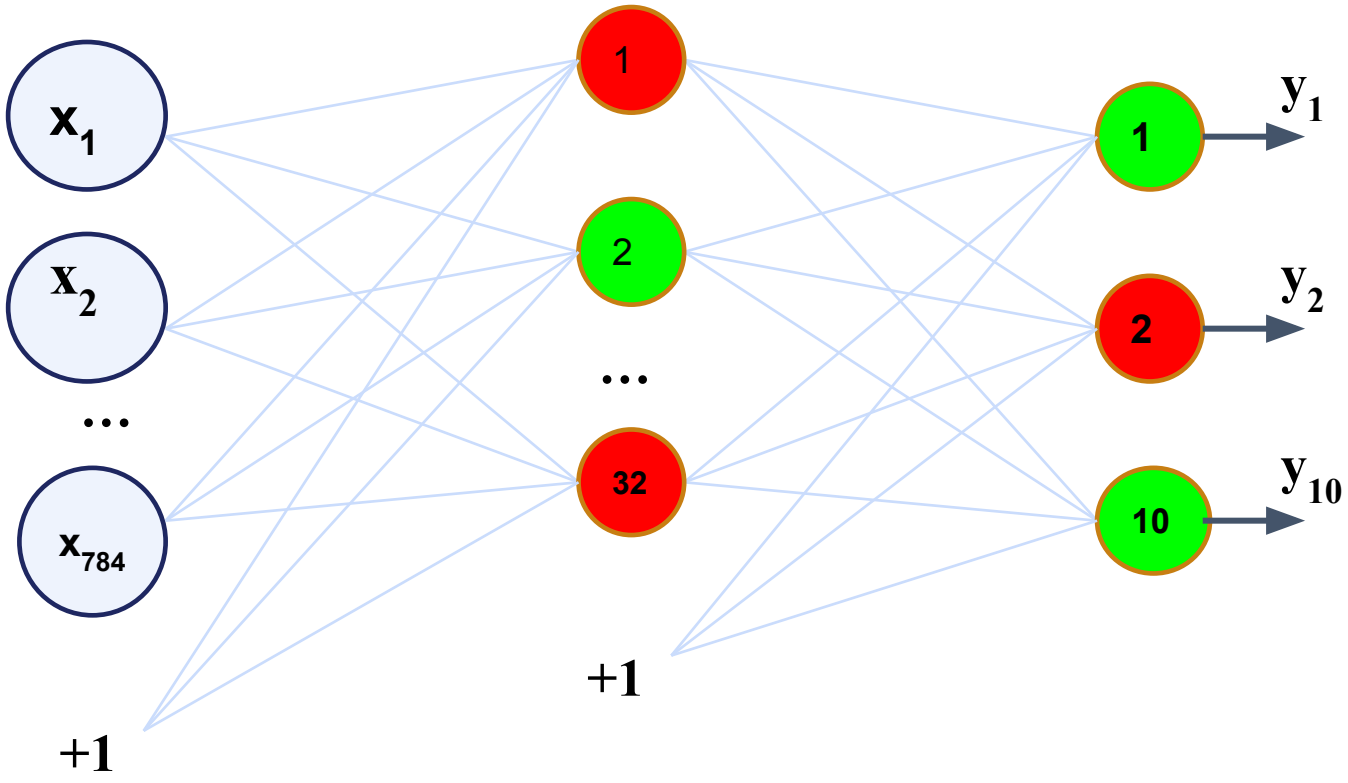
с L2-регуляризацией:
optim.Adam(params=model.parameters(), lr=0.01, weight_decay=0.001)



Dropout



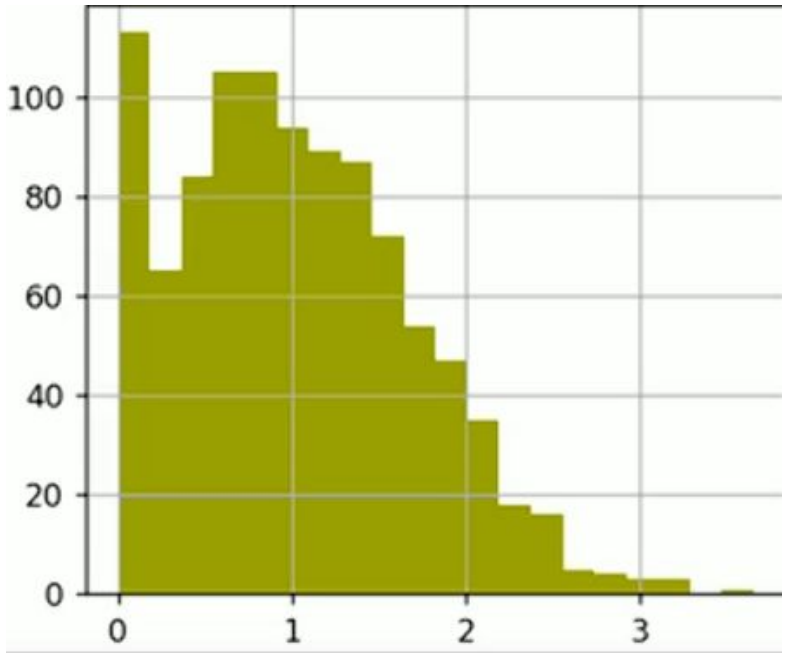
Dropout



Batch Normalization

Relu может резко обнулять значения, что приводит и резкому изменению распределения

w = [0.5, 0.5, 1.0]



w = [-0.5, 0.25, 1.0]

